GPU Self-Collision

via Spatial Hash Grid

P(F) Shape Matching PBD | CUDA GPU Implementation Plan

1. Context

The GPU shape matching PBD simulation currently handles floor collision and knife (CUSA) collision, but has no self-collision detection between particles. After debulking removes large numbers of particles, the remaining structure's shape matching constraints weaken, leading to self-intersection artifacts. A GPU-based spatial hash grid with PBD position correction is needed to prevent particle overlap.

2. Approach: Counting Sort Hash Grid + PBD Correction

Core Idea

1. Each Jacobi iteration: build spatial hash grid via counting sort
2. Each particle queries 3x3x3 neighborhood: find overlapping pairs
3. PBD-style correction: overlap -> atomicAdd into existing d_deltas/d_delta_counts, averaged together with shape matching in apply_kernel
4. Floor + knife hard constraints applied last (unchanged)

Modified step() Pipeline

```
for iter in [0, num_iterations):
    clear_deltas_kernel
    cluster_solve_kernel            // shape matching constraints
    if self_collision_enabled:
        build_hash_grid()           // 3 kernels + CUB radix sort
        self_collision_kernel       // query neighbors + atomicAdd corrections
    apply_kernel                    // merge shape matching + self-collision
    collision_kernel                // floor + knife hard constraints
```

Key design: self-collision corrections accumulate into d_deltas (shared with shape matching), unified averaging in apply_kernel. Does not directly modify predicted_pos.

3. Implementation Steps

Step 1: New GPU Data Structures

Add to ShapeMatchingGPUSolver (shape_matching_gpu.h):

```
// Self-collision hash grid buffers
uint32_t* d_cell_hash = nullptr;
uint32_t* d_particle_index = nullptr;
uint32_t* d_cell_hash_sorted = nullptr;
uint32_t* d_particle_index_sorted = nullptr;
uint32_t* d_cell_start = nullptr;       // [TABLE_SIZE]
uint32_t* d_cell_end = nullptr;         // [TABLE_SIZE]
void* d_cub_temp = nullptr;
size_t cub_temp_bytes = 0;
static const int HASH_TABLE_SIZE = 32768;  // 2^15
```

Add to SimConfigGPU:

```
float self_collision_radius;      // particle collision radius, ~0.012
float self_collision_stiffness;   // PBD stiffness, 0.5-1.0
int self_collision_enabled;       // toggle on/off
```

Step 2: GPU Memory Allocation

Allocate hash grid buffers at the end of init() (N particles + TABLE_SIZE cells). Free in cleanup(). Query CUB temp storage size via cub::DeviceRadixSort::SortPairs(nullptr, cub_temp_bytes, ...).

Step 3: Three New CUDA Kernels

Kernel 1: compute_cell_hash_kernel

```
// Per particle:
if (removed[i]) { hash = TABLE_SIZE; }  // sentinel -> sorted to end
else {
    ix = floor(pos.x / cell_size);
    iy = floor(pos.y / cell_size);
    iz = floor(pos.z / cell_size);
    hash = (ix*73856093 ^ iy*19349663 ^ iz*83492791) & (TABLE_SIZE-1);
}
cell_hash[i] = hash;
particle_index[i] = i;
```

Kernel 2: find_cell_bounds_kernel

```
// Per sorted element i:
if (i == 0 || sorted_hash[i] != sorted_hash[i-1])
    cell_start[sorted_hash[i]] = i;
if (i == N-1 || sorted_hash[i] != sorted_hash[i+1])
    cell_end[sorted_hash[i]] = i + 1;
```

Kernel 3: self_collision_kernel

```
// Per particle i:
if (removed[i] || fixed[i]) return;
(ix, iy, iz) = cell coords of pred_pos[i]
for each of 27 neighbor cells (3x3x3 stencil):
    h = hash(cx, cy, cz) & (TABLE_SIZE-1)
    for k in cell_start[h] .. cell_end[h]:
        j = sorted_index[k]
        if (j <= i || removed[j]) continue;  // avoid double-count
        dist = length(pred_pos[i] - pred_pos[j])
        if (dist < 2*radius && dist > 1e-8):
            overlap = 2*radius - dist
            dir = (pred_pos[i] - pred_pos[j]) / dist
            corr = stiffness * 0.5 * overlap * dir
            atomicAdd(&deltas[i], +corr)
            atomicAdd(&deltas[j], -corr)
            atomicAdd(&delta_counts[i], 1)
            atomicAdd(&delta_counts[j], 1)
```

Step 4: build_hash_grid() Host Function

```
void ShapeMatchingGPUSolver::build_hash_grid() {
    int N = d_particles.N;
    int blocks = (N + 255) / 256;

    // 1. Compute per-particle cell hash
    compute_cell_hash_kernel<<<blocks, 256>>>(
        d_particles.predicted_pos, d_particles.removed,
        d_cell_hash, d_particle_index,
        config.self_collision_radius * 2.0f, N);

    // 2. Sort by hash (CUB radix sort, only 15 bits)
    cub::DeviceRadixSort::SortPairs(
        d_cub_temp, cub_temp_bytes,
        d_cell_hash, d_cell_hash_sorted,
        d_particle_index, d_particle_index_sorted,
        N, 0, 15);

    // 3. Find cell boundaries
    cudaMemset(d_cell_start, 0xFF, HASH_TABLE_SIZE * sizeof(uint32_t));
    cudaMemset(d_cell_end, 0xFF, HASH_TABLE_SIZE * sizeof(uint32_t));
    find_cell_bounds_kernel<<<blocks, 256>>>(
        d_cell_hash_sorted, d_cell_start, d_cell_end, N);
}
```

Step 5: Integration into step()

Insert build_hash_grid() + self_collision_kernel between cluster_solve_kernel and apply_kernel in the Jacobi iteration loop. Self-collision corrections flow through the same d_deltas/d_delta_counts path as shape matching, ensuring consistent Jacobi averaging.

Step 6: Configuration Parameters

```
// In main_gpu_viewer.cpp:
gpu_solver.config.self_collision_radius = 0.012f;
gpu_solver.config.self_collision_stiffness = 0.8f;
gpu_solver.config.self_collision_enabled = 1;
```

particle_radius = 0.012 gives collision distance 2r = 0.024, which is less than the minimum neighbor spacing ~0.03 in cube32.msh. This ensures self-collision does NOT trigger in the rest state, only activating when deformation brings particles closer than their natural spacing.

4. Performance Budget

Operation	Per Iteration	4 Iter/Step
compute_hash	0.02 ms	0.08 ms
CUB sort (20k)	0.05 ms	0.20 ms
find_bounds	0.03 ms	0.12 ms
self_collision	0.10 ms	0.40 ms
Total	**~0.2 ms**	**~0.8 ms**

Current step time is ~2ms. With self-collision: ~3ms. Still achieves >300 FPS headless, >100 FPS with viewer. Optimization: run self-collision only on iterations 0 and 2 (every other), halving cost to ~0.4ms.

5. Files to Modify

File	Changes
solver/shape_matching_gpu.h	Hash grid buffers + config fields
solver/shape_matching_gpu.cu	3 new kernels + build_hash_grid() + step() + init()/cleanup()
main_gpu_viewer.cpp	Set self_collision config parameters
CMakeLists.txt	CUB include path (CUDA 11.8 ships CUB, usually not needed)

6. Verification

1. Basic test: cube32 free fall (disable fixed particles), verify no self-intersection
2. Debulking test: after removing large sections, verify remaining structure maintains no self-intersection
3. Performance test: pf_sim_gpu headless, compare FPS with/without self-collision
4. Correctness: in rest state, self-collision should NOT trigger (2*radius < min_spacing)

7. Key Design Decisions

Why accumulate into d_deltas?

The existing Jacobi iteration averages multiple constraint contributions via delta_counts. Self-collision is just another constraint. Accumulating into the same buffer lets apply_kernel naturally blend shape matching (keep shape) with self-collision (prevent overlap). Direct modification of predicted_pos would fight with shape matching corrections.

Why rebuild grid every iteration?

Positions change after each apply_kernel, so the grid from iteration 0 would be stale by iteration 3. Grid build is cheap (~0.1ms). As optimization, build only on iterations 0 and 2.

Why counting sort (CUB) over atomic insertion?

Counting sort + CUB radix sort produces a perfectly compact, cache-friendly structure with zero hash collisions. CUB radix sort at 20k elements takes ~0.05ms. Atomic insertion requires collision resolution and has worse worst-case.

Handling removed particles

compute_cell_hash_kernel assigns hash = TABLE_SIZE (sentinel) to removed particles. CUB sorts these to the end, so they never appear in any valid cell range.